

XÂY DỰNG HỆ THỐNG DATA LAKEHOUSE THEO DÕI THỊ TRƯỜNG CHỨNG KHOÁN HOSE

Bùi Huyền Mi, Bùi Lê Huy¹

¹VTIT

Email: huybuile2004@gmail.com

Abstract—Báo cáo trình bày một nền tảng Lakehouse hợp nhất dữ liệu giá, hồ sơ doanh nghiệp, chỉ số thị trường, tin tức và giao dịch trong phiên cho phạm vi cổ phiếu niêm yết trên HOSE. Hệ thống kết hợp pipeline theo lô chạy sau giờ giao dịch với pipeline thời gian thực từ DNSE; dữ liệu được tổ chức qua ba tầng Bronze-Silver-Gold dưới dạng Parquet trên MinIO, sau đó nạp vào ClickHouse để phục vụ truy vấn phân tích. Airflow điều phối các tác vụ, dbt xây dựng mart chỉ báo kỹ thuật, Kafka tiếp nhận tick khớp lệnh, PostgreSQL lưu watchlist và cấu hình cảnh báo, còn FastAPI cung cấp dữ liệu cho dashboard React. Cơ chế kiểm soát chất lượng gồm kiểm tra schema, miễn giá trị, khóa nghiệp vụ, độ tươi và đối soát số dòng/tập khóa giữa các tầng, áp dụng riêng cho cả lớp dữ liệu theo lô và lớp dữ liệu thời gian thực. Hai pipeline thời gian thực độc lập — tổng hợp VWAP từ tick khớp lệnh và nén một phút từ kênh riêng của DNSE — được đối soát chéo với nhau để tự phát hiện sai lệch giữa hai nguồn, thay vì tin tuyệt đối vào một luồng duy nhất. Cảnh báo kỹ thuật hỗ trợ cả điều kiện ngưỡng tĩnh và điều kiện *crossing* có trạng thái, chỉ kích hoạt khi chỉ báo vừa cắt qua ngưỡng, cùng cơ chế gộp thông báo và cooldown để giảm nhiễu. Kết quả kiểm chứng cho thấy luồng dữ liệu theo lô đạt độ nhất quán cao qua đối soát ba tầng ở cả mức số dòng và mức tập khóa nghiệp vụ, trong khi bộ kiểm tra chất lượng cho lớp thời gian thực đã tự phát hiện một sai lệch đáng kể giữa hai nguồn dữ liệu độc lập, cho thấy giá trị thực tiễn của việc xây dựng quality gate riêng cho dữ liệu streaming. Báo cáo trình bày trung thực các hạn chế còn lại, bao gồm độ phủ dữ liệu thời gian thực chưa toàn diện; riêng tin tức, bước thu thập và chuẩn hóa Silver đã chạy hằng ngày trong DAG sản xuất, còn bước liên kết thực thể và phân tích cảm xúc để lên Gold được xác định là hướng triển khai trong tuần tiếp theo.

Index Terms—Lakehouse, HOSE, Airflow, MinIO, ClickHouse, Kafka, dbt, Bronze-Silver-Gold, VWAP, data quality, cảnh báo thời gian thực.

I. GIỚI THIỆU CHUNG

Thông tin trên thị trường chứng khoán thay đổi theo nhiều nhịp khác nhau. Giá đóng cửa và chỉ báo kỹ thuật thường được cập nhật sau mỗi phiên;

hồ sơ doanh nghiệp thay đổi chậm; trong khi tick khớp lệnh và nén phút phát sinh liên tục trong giờ giao dịch. Nếu chỉ lưu dữ liệu theo lô, hệ thống không phản ánh được diễn biến trong phiên. Nếu chỉ ưu tiên streaming, hệ thống lại thiếu lịch sử ổn định để tính toán, kiểm chứng và tái xử lý. Bài toán đặt ra là xây dựng một nền tảng vừa bảo toàn dữ liệu gốc, vừa cung cấp truy vấn nhanh, đồng thời diễn giải rõ dữ liệu nào là cuối ngày và dữ liệu nào đang cập nhật trong phiên. Dữ liệu văn bản từ báo chí được tách khỏi phạm vi hiện tại để ưu tiên hoàn thiện độ đúng của dữ liệu giá.

Phạm vi được giới hạn ở sàn HOSE nhằm bảo đảm tính nhất quán xuyên suốt. Phạm vi dữ liệu gồm 403 mã cổ phiếu, VNINDEX và VN30. Các mục tiêu chính gồm: (i) hợp nhất dữ liệu nhiều nguồn và nhiều tần suất; (ii) chuẩn hóa qua kiến trúc Bronze-Silver-Gold; (iii) điều phối tự động, có khả năng chạy lại và kiểm tra lỗi; (iv) cung cấp giá cuối ngày, nén phút, VWAP, chỉ báo kỹ thuật, watchlist và cảnh báo; (v) duy trì khả năng quan sát bằng dashboard vận hành, log, quality report và bản sao lưu.

Điểm mới của giải pháp nằm ở việc tách lớp lưu trữ lâu dài khỏi lớp phục vụ, kết hợp hai đường dữ liệu độc lập cho cùng diễn biến trong phiên, và dùng đối soát chéo để phát hiện sai lệch. Cảnh báo không chỉ dựa trên ngưỡng tĩnh mà còn hỗ trợ sự kiện *crossing*, tức chỉ kích hoạt khi chỉ báo vừa cắt qua ngưỡng. Phạm vi quét có thể là một mã, toàn HOSE hoặc watchlist, nhờ đó giảm nhiễu so với việc mặc định theo dõi toàn thị trường.

II. NỘI DUNG VÀ PHƯƠNG PHÁP

A. Kiến trúc tổng thể

Hạ tầng chính gồm 14 dịch vụ Docker Compose, chia thành bốn nhóm. Nhóm lưu trữ gồm MinIO, ClickHouse và PostgreSQL. Nhóm điều phối gồm Airflow webserver và Airflow scheduler. Nhóm thời gian thực gồm Zookeeper, Kafka, DNSE producer, consumer VWAP và consumer nén phút.

TABLE I
Nguồn dữ liệu và đích phục vụ

Nguồn	Nhịp cập nhật	Nội dung và đích
KBS	Sau giờ giao dịch	OHLCV ngày, danh sách HOSE, hồ sơ doanh nghiệp; phục vụ dimension và fact giá ngày
VCI	Sau giờ giao dịch	VNINDEX, VN30; phục vụ tổng quan thị trường
DNSE	Trong giờ giao dịch	Tick khớp lệnh, nến một phút; phục vụ VWAP, biểu đồ trong phiên và cảnh báo
Nguồn báo	Dự kiến	VnExpress, Vietstock, CafeF; chưa thuộc phạm vi hoàn thành

Nhóm phục vụ và quan sát gồm Superset, Grafana, Alert Engine và dịch vụ xử lý ngôn ngữ. FastAPI và frontend React được khởi chạy riêng trong môi trường cục bộ (uvicorn và vite), không phải container Docker Compose; dbt cũng không có service riêng mà được thực thi như tác vụ Bash trong môi trường Airflow.

Kiến trúc tuân theo mô hình Lakehouse [1]: Parquet trên MinIO là lớp dữ liệu có thể truy vết và tái xử lý; ClickHouse là lớp truy vấn phân tích; PostgreSQL quản lý trạng thái cập nhật thường xuyên như watchlist và rule cảnh báo. Cách tách này tránh phụ thuộc hoàn toàn vào một cơ sở dữ liệu phục vụ, đồng thời vẫn giữ hiệu năng phù hợp cho dashboard.

B. Nguồn dữ liệu và tần suất thu thập

Bảng I tổng hợp các nguồn nằm trong phạm vi hiện tại. Dữ liệu giá ngày, danh sách mã và hồ sơ doanh nghiệp được lấy qua thư viện vnstock với nguồn KBS; dữ liệu VNINDEX/VN30 dùng nguồn VCI. DNSE WebSocket cung cấp tick khớp lệnh và nến một phút đã đóng. VnExpress, Vietstock và CafeF mới nằm trong kế hoạch mở rộng, chưa được tính vào kết quả thực hiện.

Phạm vi hiện tại và kế hoạch cho tin tức. URL registry, Bronze và Silver cho tin tức đã chạy hàng ngày trong DAG sản xuất: registry lưu trạng thái từng URL để chỉ tải bài mới; Bronze lưu nội dung gần nguyên trạng; Silver chuẩn hóa URL, tiêu đề, nội dung, nguồn, danh mục và thời điểm xuất bản. Phần còn thiếu để hoàn thành chuỗi xử lý là bước sau Silver: entity linking liên kết bài viết với ticker và mô hình ba nhãn tích cực-trung lập-tiêu cực để lên Gold. Hai bước này đã có module thử nghiệm riêng lẻ, dự kiến hoàn thiện và nối vào DAG trong tuần tiếp theo; vì chưa tự động và chưa ổn định, kết quả tin tức không được dùng làm căn cứ đánh giá và không được tính trong Bảng III.

Xử lý OHLCV và danh sách niêm yết. Danh sách mã được lấy theo exchange và chuẩn hóa về chữ hoa trước khi trở thành danh sách mã cần thu thập. Mỗi request OHLCV trả về thời gian, open, high, low, close, volume; pipeline bổ sung ticker, nguồn, thời điểm ingest và partition date. Tùy chọn --skip-existing giúp DAG không gọi lại những partition đã có, đặc biệt hữu ích khi chạy lại sau lỗi giữa chừng. Silver ép kiểu ngày/giá/khối lượng, loại dòng thiếu khóa, kiểm tra quan hệ $high \geq open, close, low$ và chỉ giữ mã thuộc HOSE. Profile listing cung cấp tên doanh nghiệp, exchange, ngành và số cổ phiếu lưu hành; các trường thiếu được chuẩn hóa về giá trị rỗng có kiểm soát thay vì làm hỏng toàn bộ dimension.

Xử lý chỉ số thị trường. VNINDEX và VN30 được khai báo tường minh trong cấu hình nguồn. Ingestion ghi raw theo từng index code; Silver chỉ chấp nhận đúng hai mã cấu hình, chuẩn hóa OHLC điểm số và volume/value. Khi tạo Gold, pipeline nối chỉ số với dim_index, tính biến động điểm, phần trăm thay đổi, SMA20, RSI14 và độ rộng thị trường. Độ rộng được xây từ số mã tăng, giảm, đứng giá trong tập HOSE; riêng VN30 sử dụng danh sách thành phần lấy tại thời điểm chạy. Việc lọc index từ Silver ngăn dữ liệu HNXINDEX/UPCOMINDEX cũ quay trở lại Gold.

Crawler nguồn báo (đang chạy). Crawler duyệt trang chuyên mục để thu URL ứng viên, tải trang chi tiết và trích tiêu đề, mô tả, nội dung, tag, thời gian xuất bản. URL được chuẩn hóa trước khi tra registry; bài thành công được đánh dấu hoàn tất, lỗi tạm thời được lên lịch retry và một URL lỗi không được phép chặn các URL còn lại. Hai mốc crawl_at và processed_at dùng để đo độ trễ. Bước này đã tham gia DAG sản xuất hàng ngày; phần chưa hoàn thành và chưa được đánh giá là entity linking và phân tích cảm xúc phía sau, dự kiến triển khai trong tuần tiếp theo.

C. Tổ chức Bronze-Silver-Gold

Bronze giữ dữ liệu thô để kiểm tra nguồn và chạy lại. OHLCV được phân vùng theo ticker/year/month/day; profile theo dataset và ngày ingest; index theo mã chỉ số và ngày. Silver chuẩn hóa kiểu dữ liệu, tên cột, mã giao dịch, thời gian và khóa nghiệp vụ. OHLCV Silver được gom theo ticker/year/month, giúp đọc lịch sử một mã mà không quét toàn bộ thị trường.

Gold trong phạm vi đánh giá sử dụng star schema gồm bốn dimension: dim_date, dim_stock, dim_sector, dim_index; cùng các fact giá ngày, chỉ số, nến phút, VWAP và sự kiện cảnh báo. Fact theo ngày được partition theo tháng; fact theo phút được partition theo ngày giao dịch. Sort key bắt đầu bằng ticker rồi đến

TABLE II
Data contract qua ba tầng

Tầng	Trách nhiệm	Kiểm soát chính
Bronze	Bảo toàn payload và metadata ingest	Nguồn, thời điểm crawl, partition, khả năng đọc Parquet
Silver	Chuẩn hóa schema và khóa nghiệp vụ	Required column, null, miễn giá trị, uniqueness, HOSE scope
Gold	Tổ chức grain phục vụ và chỉ báo	Dimension key, fact key, freshness, row/key reconciliation
Serving	Truy vấn nhanh và tổng hợp theo yêu cầu	Ngày snapshot, nguồn live, trạng thái thị trường, latency

thời gian, phù hợp với truy vấn chi tiết một mã. Sau khi nạp ClickHouse, các bảng Gold được xuất lại thành Parquet trên MinIO; tại thời điểm đánh giá, Bronze có 3.351 object/14 MiB, Silver có 406 object/2,8 MiB, Gold có 37 object/1,5 MiB và bucket backup có 87 object/9,4 MiB. Tin tức cũng có Bronze/Silver trong cùng bộ object trên nhưng chưa lên Gold, nên không được tính là sản phẩm hoàn chỉnh trong phạm vi đánh giá này.

Bảng II mô tả data contract của từng tầng. Nguyên tắc quan trọng là mỗi tầng chỉ nhận trách nhiệm mà tầng đó có thể kiểm chứng. Bronze không sửa số liệu nguồn; Silver chịu trách nhiệm chất lượng bản ghi; Gold chịu trách nhiệm grain, quan hệ dimension-fact và khả năng phục vụ truy vấn. Nhờ vậy, một lỗi nghiệp vụ ở Gold có thể được truy ngược tới Silver và raw partition tương ứng mà không phải crawl lại ngày.

Thiết kế partition cân bằng hai mục tiêu: giảm lượng dữ liệu phải đọc và tránh tạo quá nhiều file nhỏ. Bronze OHLCV phân vùng theo ticker/year/month/day (ngày ingest), đảm bảo mỗi lần chạy lại không ghi đè partition cũ và có thể truy ngược bất kỳ phiên ingest nào. Tuy nhiên, với khoảng 500 mã được ingest mỗi ngày, số lượng file Bronze OHLCV nhanh chóng đạt hàng nghìn object nhỏ trên MinIO, mỗi file chỉ chứa vài chục dòng (khoảng 4 KiB), tạo ra *small file problem* điển hình: chi phí metadata listing và số lượng read request tăng tuyến tính thay vì tăng theo lượng dữ liệu thực. Silver xử lý vấn đề này bằng cách hợp nhất toàn bộ lịch sử một mã vào một file duy nhất per tháng (ticker/year/month), giảm đáng kể số object cần đọc khi transform. Gold fact theo thời gian vì dashboard quét nhiều mã trong cùng phiên. Dimension dùng snapshot date để bảo tồn ảnh chụp tại thời điểm xuất. Với ClickHouse, partition không thay thế sort key: partition hỗ trợ quản lý vòng đời và nạp lại theo ngày/tháng, còn ORDER BY quyết định khả năng bỏ qua block khi lọc ticker và thời gian.

D. Pipeline theo lô và mô hình dbt

DAG stock_lakehouse_daily chạy lúc 18:00 từ thứ Hai đến thứ Sáu, gồm bốn nhánh ingest song song: giá, profile, index và tin tức; mỗi nhánh chuyển sang Silver tương ứng, sau đó hội tụ tại quality gate. Nếu dữ liệu hợp lệ, hệ thống tạo/cập nhật schema, nạp Gold, đối soát, chạy dbt, xuất Parquet, tạo snapshot frontend, kiểm tra cảnh báo và sao lưu. Ba nhánh giá, profile, index đi hết đường này tới Gold và được đánh giá trong báo cáo; nhánh tin tức cũng chạy thật trong DAG nhưng dừng lại ở Silver – bước entity linking và phân tích cảm xúc để lên Gold chưa được nối vào DAG, nên chưa thuộc kết quả đánh giá.

fact_daily_price chứa OHLCV, giá trị giao dịch, số cổ phiếu lưu hành, vốn hóa, biến động giá, SMA20, EMA12/26, MACD, RSI14, Bollinger Bands, trung bình khối lượng và các cờ quá mua/quá bán/breakout. Mart dbt fact_daily_price_indicators dùng chiến lược incremental delete_insert và cửa sổ nhìn lại 35 ngày. Thiết kế này giữ đúng các hàm window ở biên cập nhật nhưng không phải tái tạo toàn bộ mart sau mỗi phiên.

Vai trò của dbt trong hệ thống được giới hạn ở tầng **transform**: tính các chỉ báo kỹ thuật bằng window function của ClickHouse SQL (SMA, RSI, Bollinger Bands), nơi cú pháp SQL ngắn gọn và dễ kiểm toán hơn Python tương đương. Kiểm soát chất lượng dữ liệu được tách ra thành module Python riêng (src/quality/) thay vì dùng dbt test, vì hai lý do chính. Thứ nhất, dbt test chỉ nhìn thấy một layer (Gold/ClickHouse) trong khi hệ thống cần kiểm tra *cross-layer*: tỷ lệ mất dòng Bronze→Silver, đối sánh tập khóa Silver→Gold và freshness SLA theo ngày làm việc – các kiểm tra này đòi hỏi đọc cả Parquet local lẫn ClickHouse trong cùng một lần chạy. Thứ hai, quality gate cần chặn DAG trước khi dữ liệu vào ClickHouse, không phải sau; dbt test chạy sau dbt run nên không thể đóng vai trò cổng kiểm soát đầu vào. Hai công cụ bổ sung cho nhau: Python gate kiểm tra tính toàn vẹn dữ liệu trước khi load, dbt đảm nhận transform và dbt test xác minh ràng buộc nghiệp vụ sau khi mart đã được tạo.

Grain của fact_daily_price là một ticker trong một ngày giao dịch; grain của fact_market_index là một chỉ số trong một ngày; grain của fact_intraday_ohlc và fact_realtime_vwap là một ticker trong một phút; grain của fact_alert_event là một lần rule được kích hoạt cho một ticker và kênh gửi. Việc công bố grain trước khi viết loader giúp xác định khóa duy nhất, lựa chọn partition và viết reconciliation đúng đối tượng.

Loader Gold không xóa các bảng do service thời gian thực sở hữu. Dimension nhỏ được tái tạo hoàn toàn; fact batch dùng watermark và drop/reinsert partition tháng bị ảnh hưởng; VWAP, nến phút, alert event và alert state được bảo vệ khỏi migration batch. Với rolling indicator, pipeline đọc đủ lịch sử đầu vào nhưng chỉ ghi lại các tháng nằm trong cửa sổ tái xử lý. Cách làm này tránh mất phần đầu tháng khi một lệnh DROP PARTITION xóa cả tháng nhưng tập dữ liệu ghi lại chỉ chứa vài ngày cuối.

Tác vụ backfill nến phút chạy như một nhánh phụ vì phụ thuộc API cộng đồng có giới hạn request. Nhánh này không chặn việc xuất Gold và snapshot dashboard. Quality realtime dùng TriggerRule.ALL_DONE, do đó vẫn tạo báo cáo trên dữ liệu sẵn có kể cả khi backfill thất bại. Cách wiring này ngăn một lỗi rate limit bên ngoài làm dừng toàn bộ pipeline chính.

E. Pipeline thời gian thực và VWAP

Luồng tick bắt đầu từ DNSE WebSocket. Producer chuẩn hóa message thành ticker, thời gian giao dịch, giá và khối lượng rồi publish lên Kafka. ClickHouse Kafka Engine cùng Materialized View ghi dữ liệu vào realtime_trade_ticks_raw. Consumer chu kỳ 15 giây tổng hợp theo phút để tạo OHLC, VWAP phút, VWAP lũy kế phiên, volume/value và độ lệch giá so với VWAP.

Song song, service dnse-ohlc-consumer đăng ký kênh ohlc_closed.1, nhận nến một phút đã hoàn tất và ghi vào fact_intraday_ohlc. Nến được buffer theo lô nhỏ, lưu bản Bronze local và insert vào ClickHouse bằng ReplacingMergeTree(ingested_at) để xử lý bản ghi cập nhật. API có thể tổng hợp nến 1 phút thành 5/15/30 phút hoặc 1 giờ bằng toStartOfInterval, argMin, argMax, min, max và sum; không cần duy trì nhiều bảng vật chất hóa trùng lặp.

Dashboard phân biệt rõ hai ngữ nghĩa. Nến ngày và tổng quan cơ sở dùng phiên EOD gần nhất; trong giờ giao dịch, giá live DNSE được phủ lên giá tham chiếu khi có dữ liệu. Sau khi đóng cửa, hệ thống giữ phiên hoàn tất gần nhất, kèm ngày dữ liệu và trạng thái thị trường thay vì gắn nhãn realtime sai.

F. Cảnh báo và kênh thông báo

Rule được lưu trong PostgreSQL, sự kiện gửi được ghi vào ClickHouse. Engine hỗ trợ 14 điều kiện: giá trên/dưới ngưỡng, stop-loss, take-profit, RSI trên/dưới ngưỡng, phá Bollinger Band, lệch VWAP, đột biến khối lượng phút, breakout 20 phút, RSI crossing lên/xuống và VWAP crossing

lên/xuống. Rule có thể áp dụng cho một ticker, WATCHLIST hoặc ALL.

Các điều kiện crossing cần giá trị chu kỳ trước. Bảng fact_alert_rule_state sử dụng ReplacingMergeTree, khóa (alert_id,ticker), lưu metric gần nhất bằng argMax. Engine cập nhật trạng thái mỗi chu kỳ, kể cả khi chưa trigger, nhờ đó chỉ gửi khi thực sự xuất hiện sự kiện cắt ngưỡng. Cooldown ngăn lặp thông báo; các mã kích hoạt cùng chu kỳ được gom thành digest tối đa 25 dòng mỗi điều kiện. Telegram phù hợp cảnh báo trong phiên, còn email phù hợp tổng hợp EOD. Lịch sử hiện ghi nhận cả trạng thái sent, send_failed và channel_not_configured, hỗ trợ kiểm tra khả năng giao nhận thay vì chỉ lưu điều kiện kích hoạt.

Một chu kỳ alert gồm sáu bước: đọc rule active; mở rộng ALL/WATCHLIST; lấy snapshot chỉ báo mới nhất theo tập ticker cần thiết; đọc trạng thái cũ cho rule crossing; đánh giá điều kiện và cooldown; gom nhóm theo (user_id,channel), gửi rồi ghi event/state. Giá EOD được dùng làm fallback, nhưng nếu có nến phút của ngày hiện tại thì close intraday ghi đè để stop-loss, take-profit và breakout không so sánh hai phiên khác nhau. VWAP luôn lọc đúng today() và nguồn DNSE, ngăn phiên cũ kích hoạt cảnh báo ngoài ý muốn.

PostgreSQL phù hợp với rule/watchlist vì cần insert, update, delete giao dịch và unique constraint theo người dùng. ClickHouse phù hợp với event/state history vì phần lớn thao tác là append và truy vấn thống kê. Tách hai loại workload giúp form cấu hình phản hồi đơn giản nhưng lịch sử hàng trăm sự kiện vẫn được tổng hợp nhanh theo ngày, condition và delivery status.

G. Kiểm soát chất lượng, quan sát và an toàn vận hành

Quality gate Silver kiểm tra cột bắt buộc, null, giá dương, quan hệ OHLC, volume không âm, mã sàn hợp lệ, độ dài nội dung và uniqueness. Reconciliation đối chiếu Bronze-Silver-Gold bằng cả số dòng và anti-join tập khóa, tránh trường hợp số lượng bằng nhau nhưng bản ghi bị thay thế sai. Freshness được đo theo ngày làm việc thay vì ngày lịch.

Bộ kiểm tra realtime gồm bảy kỳ vọng: không trùng khóa nến phút; nến nằm trong giờ HOSE; quan hệ OHLC hợp lệ; freshness trong phiên; tỷ lệ phủ mã; không trùng khóa VWAP; và đối soát volume giữa tick/VWAP với nến phút. Grafana theo dõi độ trễ ingest, số sự kiện cảnh báo, freshness và số dòng Gold; Superset phục vụ phân tích bổ sung; pipeline monitor đọc trạng thái task từ Airflow. CI gồm lint/test Python, dbt parse, import DAG, kiểm tra Docker Compose và build frontend. Secret hỗ trợ biến <NAME>_FILE; backup dùng pg_dump cho

TABLE III
Kết quả dữ liệu và kiểm chứng

Đối tượng	Số lượng	Phạm vi/ghi chú
Mã cổ phiếu HOSE	403	Không gồm HNX/UPCOM
fact_daily_price	13.339	166 ngày giao dịch
fact_market_index	68	VNINDEX và VN30
fact_intraday_ohlcv	5.146	320 mã, hai ngày dữ liệu
fact_realtime_vwap	4.287	325 mã trong phiên
Raw tick DNSE	51.227	325 mã trong phiên
fact_alert_event	884	Telegram và email
Airflow	22 task	Lịch 18:00, thứ Hai-Sáu
Docker Compose	14	Batch, streaming, service, serving, quan sát
Reconciliation batch	10/10	Không thiếu/dư khóa
Quality realtime	6/7	Phát hiện lệch volume
Python test	214/214	Full suite đạt toàn bộ

TABLE IV
Luồng phục vụ dashboard

Màn hình	Nguồn phục vụ	Chỉ số chính
Tổng quan	Daily price, index, live overlay	VNINDEX/VN30, breadth, top tăng/giảm, volume/value
Chi tiết mã	Daily mart, intraday OHLCV	Nến, SMA/EMA, RSI, MACD, Bollinger, volume
Scanner	Indicator mart và nến phút	Quá mua/bán, breakout, MACD, volume spike
VWAP	Realtime VWAP	Giá cuối, VWAP phút/phiên, độ lệch, KL lũy kế
Watchlist/Alert	PostgreSQL và alert event	Mã theo dõi, rule, cooldown, delivery status
Pipeline	Airflow, ClickHouse, Kafka	Trạng thái task, row count, quality error, lag

PostgreSQL và định dạng Native nén của ClickHouse, lưu 14 ngày trên MinIO.

Mỗi quality report được lưu dạng JSON với thời điểm sinh, tổng số bản ghi, số lỗi và chi tiết từng expectation. Tác vụ reconciliation dùng --fail-on-error nên chặn Gold khi batch mất khóa hoặc quá freshness SLA. Ngược lại, realtime quality hiện mang tính chẩn đoán: task vẫn thành công về mặt điều phối nhưng report có success=false, tránh làm dữ liệu EOD hợp lệ bị ngừng công bố vì một luồng streaming không đầy đủ. Sự khác biệt này thể hiện chủ đích giữa *quality gate* và *observability check*.

CI được tách thành nhiều job để lỗi có phạm vi rõ ràng. Lint/test xác minh module Python; dbt parse kiểm tra Jinja/model graph mà không cần ClickHouse; DAG import cài Airflow và xác nhận toàn bộ task được tạo; Docker Compose validation dùng file biến mẫu không chứa secret; frontend build kiểm tra TypeScript và bundle production. Việc tách job cho phép sửa đúng lớp thay vì xem một trạng thái đỏ chung là lỗi toàn bộ hệ thống.

III. KẾT QUẢ THỰC HIỆN

Số liệu được truy vấn trực tiếp ngày 29/06/2026 và trình bày tại Bảng III. Pipeline theo lô mới nhất hoàn thành từ ingest đến export dashboard; backfill nến phút thất bại sau 24 phút 27 giây do giới hạn nguồn ngoài nhưng không làm nghẽn nhánh Gold. Tác vụ quality realtime vẫn chạy và tạo báo cáo như thiết kế.

Dashboard cung cấp các nhóm chức năng: tổng quan thị trường và độ rộng; watchlist; chi tiết cổ phiếu với nến ngày/nến phút và RSI/MACD; bộ lọc cổ phiếu; scanner kỹ thuật EOD/intraday; theo dõi VWAP, volume và độ lệch; lịch sử/cấu hình cảnh báo; cùng trang theo dõi pipeline. Màn hình tin

tức chưa được tính là chức năng hoàn thiện. API trả metadata về ngày giao dịch, nguồn và trạng thái thị trường để giao diện không trộn dữ liệu EOD với dữ liệu live.

FastAPI cung cấp endpoint cho danh sách mã, candles EOD, intraday candles, market overview, VWAP snapshot/series, technical signals, alert history, pipeline status, watchlist và rule management. Với candles intraday, API chọn ngày mới nhất của ticker, ưu tiên DNSE khi cùng phút có nhiều nguồn và dùng argMax để lấy phiên bản mới nhất. Với market overview, payload có dataMode, isRealtime, liveAsOf và số ticker live. Những trường này giúp frontend trình bày trung thực mức độ tươi thay vì suy đoán từ đồng hồ trình duyệt.

IV. ĐÁNH GIÁ HIỆU QUẢ

A. Điểm mạnh

Luồng theo lô có độ hoàn thiện cao: phạm vi HOSE nhất quán, key không trùng, freshness đạt SLA và đối soát ba tầng không mất dữ liệu. Kiến trúc cho phép tái xử lý từ Parquet, trong khi ClickHouse vẫn bảo đảm tốc độ truy vấn. Hai pipeline thời gian thực độc lập tạo điều kiện kiểm chứng chéo thay vì tin tuyệt đối vào một nguồn. Cảnh báo có cooldown, batching, watchlist và trạng thái crossing nên sát hành vì sử dụng thực tế hơn so với so sánh ngưỡng tĩnh. Hệ thống cũng đã có CI, monitoring, backup và healthcheck, giảm phụ thuộc vào thao tác thủ công.

Về hiệu quả lưu trữ, tổng dung lượng các bucket dữ liệu hiện dưới 30 MiB vì lịch sử còn ngắn; Parquet và nén cột phù hợp để mở rộng mà không làm tăng tuyến tính chi phí truy vấn dashboard. ClickHouse tách storage layout theo grain nên truy vấn một ticker/ngày hoặc một phiên không

phải đọc toàn bộ file raw. Về hiệu quả vận hành, nhánh batch mới nhất hoàn tất các bước ingest chính trong vài giây đến dưới hai phút mỗi task; dbt run 10 giây, dbt test 7 giây, export Gold 4 giây và frontend snapshot 2 giây. Backfill nền phút là ngoại lệ vì chịu rate limit nguồn ngoài.

Về khả năng kiểm chứng, 214 kiểm thử Python bao phủ transform, loader, schema, API, alert, Kafka consumer, URL registry, quality và DAG wiring. Reconciliation không chỉ so row count mà còn anti-join khóa theo hai chiều. Cách kiểm tra này mạnh hơn so sánh tổng số dòng: hai tầng có thể cùng 13.339 dòng nhưng vẫn chứa các tập ticker/date khác nhau; anti-join sẽ phát hiện trường hợp đó.

B. Hạn chế và nguyên nhân

Thứ nhất, quality realtime phát hiện session_volume từ VWAP là 9.103.410, trong khi tổng volume nền phút là 62.082.100; sai lệch 85,3% vượt ngưỡng 15%. Nguyên nhân có thể đến từ mất tick khi WebSocket reconnect, khác biệt tập mã hoặc khác biệt ngữ nghĩa volume giữa hai kênh DNSE. Kết quả này cần được điều tra trước khi dùng volume/VWAP cho quyết định có độ nhạy cao.

Thứ hai, lịch sử chưa đồng đều: bảng giá có 166 ngày giao dịch tổng thể nhưng mỗi ticker chỉ có 24–42 phiên (trung vị 34 phiên); index có 34 phiên. Dữ liệu đủ minh họa chỉ báo cơ bản nhưng chưa phù hợp cho backtest dài hạn. Thứ ba, nền phút realtime mới phủ 79,4% tổng số mã HOSE trong phiên; đây có thể gồm mã không phát sinh giao dịch, nhưng cần metric tách “đã subscribe”, “có tick” và “có nền đóng” để đánh giá chính xác.

Thứ tư, Bronze/Silver realtime vẫn có phần phụ thuộc thư mục local, trong khi mục tiêu Lakehouse yêu cầu MinIO là điểm lưu trữ bền vững xuyên suốt. Thứ năm, tin tức mới hoàn thành tới Silver trong DAG sản xuất; entity linking và phân tích cảm xúc để lên Gold chưa được nối, dự kiến triển khai trong tuần tiếp theo sau khi luồng dữ liệu giá đạt độ ổn định cần thiết. Cuối cùng, hệ thống hiện phù hợp môi trường một người dùng và localhost; mật khẩu mặc định, image gắn tag latest, API chưa có xác thực và backup nằm cùng cụm MinIO nên chưa đáp ứng triển khai công khai có tính sẵn sàng cao.

C. Hướng cải thiện

Ưu tiên tiếp theo gồm: (1) đo offset/message loss tại từng lần reconnect và thống nhất cách tính volume giữa hai kênh DNSE; (2) ghi Bronze và Silver realtime trực tiếp lên MinIO, sau đó bổ sung reconciliation theo phút/ngày; (3) backfill tối thiểu một năm giá ngày và 30–90 ngày nền phút;

(4) nối entity linking và phân tích cảm xúc tin tức vào Gold (đã có crawl, registry, Bronze, Silver); (5) bổ sung cache API, loại mock fallback gây nhầm lẫn và tối ưu pipeline status; (6) pin phiên bản image, thêm xác thực, chuyển secret sang file/secret store và sao lưu sang vị trí độc lập; (7) kiểm thử khôi phục dữ liệu định kỳ.

Hướng cải thiện lưu trữ: Apache Iceberg và compaction. Vấn đề file nhỏ ở tầng Bronze (hàng nghìn object ~4 KiB trên MinIO) là hệ quả trực tiếp của chiến lược partition theo ngày ingest nhằm bảo toàn idempotency. Một hướng xử lý trong tương lai là áp dụng **Apache Iceberg** – table format hỗ trợ ACID transactions, hidden partitioning và *automatic file compaction* – thay cho Parquet thuần. Iceberg cho phép định nghĩa partition logic độc lập với cấu trúc thư mục vật lý; compaction gom các file nhỏ lại định kỳ mà không ảnh hưởng truy vấn đang chạy. Ngoài ra, Iceberg hỗ trợ schema evolution (thêm/đổi tên cột không cần migrate toàn bộ dataset), time travel (truy vấn snapshot tại bất kỳ thời điểm nào trong quá khứ) và row-level delete – các tính năng cần thiết khi hệ thống mở rộng sang nhiều nguồn dữ liệu và yêu cầu tuân thủ quy định xóa dữ liệu. Tích hợp Iceberg trên MinIO (S3-compatible) với Spark hoặc PyIceberg là bước mở rộng tự nhiên từ kiến trúc Medallion hiện tại.

Kế hoạch xử lý tin tức và sentiment. Pipeline dự kiến thu thập bài mới từ VnExpress, Vietstock và CafeF theo chu kỳ năm phút. URL registry lưu URL chuẩn hóa, trạng thái xử lý, số lần thử và thời điểm retry để mỗi chu kỳ chỉ tải nội dung mới. Bronze trên MinIO giữ payload gần nguyên trạng và metadata nguồn; Silver làm sạch HTML, chuẩn hóa tiêu đề/nội dung/thời gian, loại bài trùng hoặc quá ngắn và kiểm tra freshness. Bước entity linking sử dụng ticker, tên đầy đủ và tên viết tắt trong dim_stock để gán một bài với một hoặc nhiều mã HOSE, kèm điểm khớp để hạn chế liên kết sai. Nội dung sau liên kết được đưa qua mô hình tiếng Việt để phân loại ba nhãn *tích cực*, *trung lập*, *tiêu cực*; đầu ra lưu thêm confidence, model version và thời điểm inference nhằm hỗ trợ tái lập kết quả. Gold sẽ tổng hợp theo ticker + news_date, gồm số bài, số nguồn, số lượng từng nhãn, điểm cảm xúc trung bình và tiêu đề nổi bật. Quality gate kiểm tra URL duy nhất, nội dung tối thiểu, tổng ba nhãn bằng số bài, ticker thuộc HOSE và độ trễ từ lúc xuất bản đến lúc phục vụ. Kết quả sau cùng sẽ bổ sung cho trang chi tiết cổ phiếu, dashboard cảm xúc thị trường và rule cảnh báo khi xuất hiện tin tiêu cực có độ tin cậy vượt ngưỡng.

V. KẾT LUẬN

Nền tảng đã xây dựng được một chuỗi xử lý tương đối đầy đủ cho thị trường HOSE: thu thập nhiều nguồn, lưu trữ ba tầng, mô hình hóa Gold, tính chỉ báo, xử lý tick/nến phút, cung cấp API/dashboard, cảnh báo qua Telegram/email, điều phối tự động và kiểm soát chất lượng. Kết quả nổi bật không chỉ là số lượng bảng hay dịch vụ, mà là khả năng truy vết và tự phát hiện sai lệch: pipeline theo lô đạt đối soát tuyệt đối, còn quality realtime chỉ ra một chênh lệch volume lớn cần xử lý. Với việc khắc phục tính nhất quán realtime, mở rộng lịch sử và hoàn thiện đường inference, hệ thống có thể phát triển từ nền tảng phân tích chạy cục bộ thành dịch vụ giám sát thị trường ổn định hơn.

Mã nguồn đi kèm bao gồm cấu hình Docker Compose, DAG Airflow, DDL ClickHouse/PostgreSQL, mô hình dbt, module Python, dịch vụ API, frontend React, bộ kiểm thử, dashboard Grafana/Superset và các script khởi tạo, sao lưu, benchmark, kiểm tra chất lượng và đối soát.

TÀI LIỆU THAM KHẢO

- [1] M. Armbrust *et al.*, "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics," in *CIDR*, 2021.
- [2] Apache Software Foundation, "Apache Parquet Documentation," [Online]. Available: <https://parquet.apache.org/docs/>. Accessed: Jun. 29, 2026.
- [3] MinIO, "MinIO Object Store Documentation," [Online]. Available: <https://min.io/docs/>. Accessed: Jun. 29, 2026.
- [4] Apache Software Foundation, "Apache Airflow Documentation," [Online]. Available: <https://airflow.apache.org/docs/>. Accessed: Jun. 29, 2026.
- [5] ClickHouse, Inc., "ClickHouse Documentation," [Online]. Available: <https://clickhouse.com/docs>. Accessed: Jun. 29, 2026.
- [6] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A Distributed Messaging System for Log Processing," in *NetDB*, 2011.
- [7] dbt Labs, "dbt Developer Hub," [Online]. Available: <https://docs.getdbt.com/>. Accessed: Jun. 29, 2026.
- [8] S. Ramírez, "FastAPI Documentation," [Online]. Available: <https://fastapi.tiangolo.com/>. Accessed: Jun. 29, 2026.